ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



NỀN TẢNG LẬP TRÌNH CHO PHÂN TÍCH
VÀ TRỰC QUAN DỮ LIỆU
Mã môn học: CO5177

BÁO CÁO TEXT

Phân loại tin tức VietnamNet bằng PhoBERT

Fine-tuning, temperature scaling và ngưỡng theo lớp

Học kỳ 261 · Năm học 2026–2027

Giảng viên: Lê Thành Sách

Thành viên: Huỳnh Thành Đạt · 2570161
Lương Minh Duy · 2570083

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Bài toán và hướng tiếp cận **1**
1.1 Mục tiêu nghiên cứu . . . 1
1.2 Quy trình thực nghiệm . . . 1

2 Dữ liệu và đặc điểm bài báo **2**
2.1 Bộ dữ liệu VietnamNet . . . 2
2.2 Sự chênh lệch giữa các chuyên mục . . . 2
2.3 Độ dài văn bản và giới hạn đầu vào . . . 2

3 Chuẩn bị dữ liệu cho PhoBERT **4**
3.1 Làm sạch và tách từ . . . 4
3.2 Chia tập và thống nhất nhãn . . . 4
3.3 Mã hóa Head–Tail . . . 4

4 Huấn luyện và kết quả phân loại **5**
4.1 Trọng số lớp và hàm mất mát . . . 5
4.2 Kiến trúc và cấu hình huấn luyện . . . 5
4.3 Diễn biến qua bảy epoch . . . 5
4.4 Kết quả trước hiệu chỉnh . . . 6
4.5 Lưu mô hình và kiểm tra suy luận . . . 7

5 Hiệu chỉnh xác suất và ngưỡng phân loại **9**
5.1 Temperature scaling . . . 9
5.2 Tìm ngưỡng cho từng chuyên mục . . . 9
5.3 Kết quả sau hiệu chỉnh . . . 10
5.4 Lưu tham số hiệu chỉnh . . . 10

6 Phân tích lỗi và kết luận **12**
6.1 Những chuyên mục còn khó phân biệt . . . 12
6.2 Giới hạn thực nghiệm . . . 12
6.3 Kết luận . . . 13

A Thực nghiệm bổ sung trên tiêu đề **14**

B Biểu đồ bổ sung **15**

C Mã nguồn và đối chiếu kết quả **16**
C.1 Môi trường và cấu hình . . . 16
C.2 Tệp thực nghiệm và kết quả . . . 16

Tài liệu tham khảo **17**

Chương 1

Bài toán và hướng tiếp cận

1.1 Mục tiêu nghiên cứu

Phân loại tin tức tiếng Việt đòi hỏi mô hình nhận biết nội dung của bài viết và phân biệt những chuyên mục có phạm vi gần nhau. Trong bộ dữ liệu VietnamNet, các nhóm như Thời sự, Pháp luật hay Kinh doanh có thể cùng đề cập đến một sự kiện nhưng nhấn mạnh những khía cạnh khác nhau. Bên cạnh đó, độ dài bài viết và số lượng mẫu giữa các chuyên mục cũng có sự chênh lệch đáng kể.

Thực nghiệm sử dụng PhoBERT Base v2 để phân loại 212.705 bài báo vào 19 chuyên mục. Tiêu đề được kết hợp với phần đầu và phần cuối của nội dung; mô hình được fine-tune bằng hàm mất mát có trọng số để tăng mức đóng góp của các lớp ít mẫu. Sau huấn luyện, temperature scaling và ngưỡng theo lớp được áp dụng nhằm điều chỉnh độ tự tin và quyết định phân loại.

Báo cáo tập trung vào ba vấn đề: cách biểu diễn bài báo dài trong giới hạn đầu vào của PhoBERT, kết quả phân loại khi dữ liệu mất cân bằng, và ảnh hưởng của calibration đến từng chuyên mục. Một thực nghiệm riêng trên 7.600 tiêu đề được trình bày ở phụ lục để bổ sung góc nhìn về việc sử dụng đặc trưng văn bản.

1.2 Quy trình thực nghiệm

Quá trình xử lý bắt đầu từ khảo sát dữ liệu, sau đó lần lượt thực hiện làm sạch, tách từ, xây dựng đầu vào và huấn luyện. Kết quả được phân tích qua các chỉ số tổng hợp, đường cong học và lỗi theo chuyên mục. Phần hiệu chỉnh được thực hiện sau khi mô hình đã hoàn tất huấn luyện.

Bước	Nội dung	Công việc thực hiện
0	Thiết lập	Kiểm tra thư viện, thiết bị và cấu hình huấn luyện.
1	Nạp dữ liệu	Đọc 19 tệp parquet, kiểm tra số bài và dữ liệu thiếu.
2	Khảo sát	Phân tích phân bố lớp và độ dài văn bản.
3	Tiền xử lý	Làm sạch, tách từ bằng PyVi và lưu kết quả.
4	Tạo dataset	Chia tập, mã hóa Head–Tail và tính trọng số lớp.
5	Huấn luyện	Fine-tune PhoBERT, theo dõi và chọn checkpoint.
6	Đánh giá	Tính accuracy, F1 và phân tích ma trận nhầm lẫn.
7	Xuất mô hình	Lưu trọng số, tokenizer và cấu hình suy luận.
8	Hiệu chỉnh	Tìm temperature và ngưỡng riêng cho từng lớp.
9	Phân tích lỗi	Xem xét lớp khó và đánh đổi precision–recall.

Bảng 1.1: Các bước thực nghiệm, tương ứng với Section 0–9 của notebook.

Chương 2

Dữ liệu và đặc điểm bài báo

2.1 Bộ dữ liệu VietnamNet

Bộ dữ liệu do Huỳnh Thành Đạt thu thập từ VietnamNet, gồm tiêu đề, nội dung, URL và chuyên mục của bài báo [1]. Dữ liệu được lưu trong 19 tệp parquet, mỗi tệp tương ứng với một chuyên mục. Khi nạp, nhãn được ánh xạ từ tên tệp sang tên chuyên mục tiếng Việt; văn bản dùng cho mô hình chỉ gồm tiêu đề và nội dung bài viết.

Tổng số bài là 212.705, trong đó 1.167 bài thiếu tiêu đề và 7 bài thiếu nội dung. Không có bài nào thiếu đồng thời cả hai trường. Vì vậy, những bài thiếu tiêu đề vẫn được giữ để khai thác nội dung còn lại; tương tự, bài thiếu nội dung được biểu diễn bằng tiêu đề. Sau tiền xử lý, toàn bộ 212.705 bài vẫn có văn bản hợp lệ.

Dữ liệu được đọc theo từng batch 20.480 dòng để hạn chế bộ nhớ sử dụng. Do thông tin về quyền phân phối lại nội dung báo chí chưa được nêu rõ trên trang dataset, repository chỉ lưu mã nguồn và kết quả phân tích; dữ liệu thô được giữ riêng.

2.2 Sự chênh lệch giữa các chuyên mục

Phần lớn chuyên mục có khoảng 12.500 bài. Hai lớp ít mẫu nhất là Dân tộc – Tôn giáo với 3.355 bài và Bảo vệ người tiêu dùng với 3.873 bài. Số bài của lớp lớn nhất gấp khoảng 3,73 lần lớp nhỏ nhất. Chênh lệch này ảnh hưởng đến cả quá trình học lẫn cách đọc kết quả: một mô hình dự đoán tốt các lớp lớn vẫn có thể đạt accuracy cao dù bỏ sót nhiều bài ở lớp nhỏ.

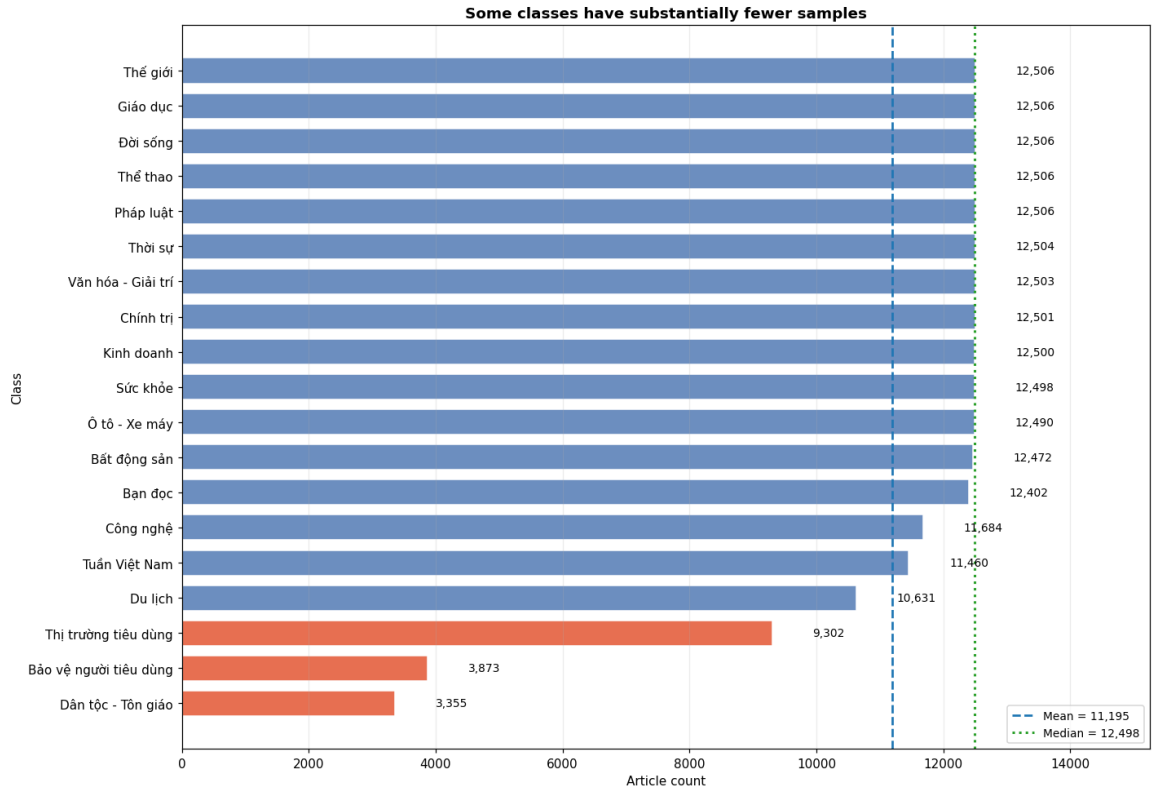
Vì vậy, macro-F1 được dùng làm tiêu chí chọn checkpoint, bên cạnh accuracy và weighted-F1. Mỗi lớp có mức đóng góp ngang nhau vào macro-F1, giúp kết quả của các chuyên mục ít mẫu được phản ánh rõ hơn.

2.3 Độ dài văn bản và giới hạn đầu vào

Độ dài trung vị của văn bản nguồn là 625 từ; phân vị 90 và 95 lần lượt là 1.218 và 1.505 từ. Có 89,9% bài dài hơn 256 từ. Trong khi đó, đầu vào của mô hình chỉ chứa tối đa 256 token, nên cần lựa chọn phần văn bản được giữ lại.

Ở bước nạp dữ liệu, phần nội dung được rút gọn bằng cách lấy tối đa 220 từ đầu và 80 từ cuối. Tiêu đề được nối với phần nội dung này trước khi làm sạch và tách từ. Cách chọn đầu–cuối nhằm giữ lại cả phần mở đầu và phần kết của bài báo trong một đầu vào ngắn hơn.

Sau xử lý, mẫu 1.000 văn bản có độ dài trung bình 246,3 token, trung vị 254 token; 46,3% mẫu vượt 256 token. Thống kê này được đo bằng tokenizer trên nội dung đã rút gọn, nên khác với tỷ lệ 89,9% tính theo số từ của bài nguồn. Biểu đồ độ dài được trình bày ở phụ lục B.



Hình 2.1: Phân bố số bài trong 19 chuyên mục.

Chuyên mục	Số bài	Tỷ lệ
Bạn đọc	12,402	5.83%
Bảo vệ người tiêu dùng	3,873	1.82%
Bất động sản	12,472	5.86%
Chính trị	12,501	5.88%
Công nghệ	11,684	5.49%
Du lịch	10,631	5.00%
Dân tộc - Tôn giáo	3,355	1.58%
Giáo dục	12,506	5.88%
Kinh doanh	12,500	5.88%
Pháp luật	12,506	5.88%
Sức khỏe	12,498	5.88%
Thế giới	12,506	5.88%
Thể thao	12,506	5.88%
Thị trường tiêu dùng	9,302	4.37%
Thời sự	12,504	5.88%
Tuần Việt Nam	11,460	5.39%
Văn hóa - Giải trí	12,503	5.88%
Ô tô - Xe máy	12,490	5.87%
Đời sống	12,506	5.88%

Bảng 2.1: Số bài và tỷ lệ của từng chuyên mục trong dữ liệu nguồn.

Chương 3

Chuẩn bị dữ liệu cho PhoBERT

3.1 Làm sạch và tách từ

Chuỗi đầu vào được tạo bằng cách nối tiêu đề với phần nội dung đầu–cuối. Văn bản sau đó được chuyển sang chữ thường, thay dấu câu bằng khoảng trắng, loại chữ số và tách từ bằng `pyvi.ViTokenizer`. Các khoảng trắng thừa được gộp lại ở cuối bước xử lý.

Stopword được giữ vì PhoBERT khai thác quan hệ ngữ cảnh trong câu [3, 4]. Tuy nhiên, việc loại chữ số cũng có thể làm mất thông tin như năm, giá tiền hoặc tỷ số thể thao. Thực nghiệm hiện tại chưa có đối chứng riêng cho bước này, nên chưa xác định được mức ảnh hưởng đến kết quả phân loại.

Tiền xử lý được chia thành các batch 512 bài, chạy với hai worker. Văn bản đã tách từ, nhãn và ánh xạ nhãn–ID được lưu vào cache để tránh xử lý lại trong các lần mở notebook tiếp theo.

3.2 Chia tập và thống nhất nhãn

Dữ liệu được chia phân tầng theo chuyên mục với tỷ lệ 85/15 và seed 42. Tập huấn luyện có 180.799 bài; tập còn lại có 31.906 bài. Các chuyên mục được sắp xếp và gán ID từ 0 đến 18. Thứ tự này được dùng nhất quán trong hàm mất mát, đầu ra mô hình, ma trận nhầm lẫn và cấu hình ngưỡng.

Tập 31.906 bài được đặt tên là test trong mã nguồn, nhưng đồng thời được dùng để đánh giá sau mỗi epoch, chọn checkpoint và tìm tham số calibration. Trong báo cáo, tập này được gọi là *tập đánh giá* để thể hiện đúng vai trò của nó. Kết quả trên tập này cần được đọc cùng giới hạn nêu ở Chương 6.

3.3 Mã hóa Head–Tail

Sau tách từ, tokenizer PhoBERT chuyển văn bản thành các subword ID. Hai vị trí trong chuỗi 256 token được dành cho token đặc biệt. Nếu văn bản có tối đa 254 token nội dung, toàn bộ chuỗi được giữ và bổ sung padding. Với bài dài hơn, đầu vào gồm 127 token đầu và 127 token cuối:

$$x = [\text{CLS}] \parallel t_{1:127} \parallel t_{L-126:L} \parallel [\text{SEP}].$$

Attention mask đánh dấu token thật bằng 1 và padding bằng 0. Input IDs, attention mask và nhãn được lưu thành tensor trong `HeadTailDataset`; tập train và tập đánh giá có cache riêng.

Nội dung vì thế được rút gọn hai lần: 220+80 từ ở bước nạp body, rồi 127+127 subword token khi mã hóa. Đầu vào cuối chỉ giữ một phần bài viết.

Chương 4

Huấn luyện và kết quả phân loại

4.1 Trọng số lớp và hàm mất mát

Để giảm ảnh hưởng của sự mất cân bằng, trọng số mỗi lớp được tính từ nhãn trong tập huấn luyện:

$$w_c = \frac{N_{\text{train}}}{K\, n_c}\, b_c, \qquad K = 19,$$

trong đó n_c là số mẫu của lớp c, còn b_c là hệ số điều chỉnh bổ sung. Dân tộc – Tôn giáo dùng hệ số 1,35; Kinh doanh và Thời sự dùng 1,15; Thị trường tiêu dùng và Đời sống dùng 1,10. Các lớp còn lại giữ hệ số 1.

Hàm mất mát là cross-entropy có trọng số. Với batch B, cách lấy trung bình của PyTorch được viết như sau:

$$\mathcal{L}_B = -\frac{\sum_{i \in B} w_{y_i} \log p_\theta(y_i \mid x_i)}{\sum_{i \in B} w_{y_i}}.$$

Trọng số cao làm lỗi ở lớp tương ứng đóng góp nhiều hơn vào loss. Các hệ số bổ sung được giữ cố định trong thực nghiệm; chưa có phép so sánh riêng để tách ảnh hưởng của chúng khỏi trọng số balanced ban đầu.

4.2 Kiến trúc và cấu hình huấn luyện

PhoBERT Base v2 được gắn đầu phân loại 19 lớp. Biểu diễn của token đầu đi qua dropout, một lớp dense với hàm tanh và lớp chiếu ra 19 logits. Cả encoder và đầu phân loại đều được cập nhật trong quá trình fine-tune.

`AutoModelForSequenceClassification` được dùng để khởi tạo kiến trúc này từ thư viện Transformers.

`WeightedTrainer` kế thừa Trainer của Transformers và thay hàm tính loss bằng cross-entropy có trọng số. Mô hình được huấn luyện bằng fused AdamW, learning rate 2×10^{-5} và weight decay 0,01. Batch mỗi GPU là 16; với hai GPU và hai bước tích lũy gradient, batch hiệu dụng toàn cục là 64.

4.3 Diễn biến qua bảy epoch

Sau mỗi epoch, mô hình được đánh giá và so sánh theo macro-F1. Checkpoint tốt nhất được nạp lại trước khi lưu mô hình cuối cùng. Quá trình huấn luyện hoàn tất đủ bảy epoch trong khoảng 38.115 giây, tương đương 10 giờ 35 phút. Tổng thời gian thực hiện notebook khoảng 11 giờ 1 phút, bao gồm chuẩn bị dữ liệu, suy luận và hiệu chỉnh.

Tham số	Giá trị
Mô hình	PhoBERT Base v2, khoảng 135 triệu tham số
Số epoch / bước cập nhật	7 / 19.775
Độ dài đầu vào	256 token
Batch mỗi GPU / tích lũy gradient	16 / 2
Thiết bị / batch hiệu dụng	2 Tesla T4 / 64
Batch đánh giá mỗi GPU	32
Learning rate / weight decay	2×10^{-5} / 0,01
Warmup / optimizer	200 bước / fused AdamW
Độ chính xác số	FP16
Đánh giá và lưu checkpoint	Sau mỗi epoch
Tiêu chí chọn checkpoint	Macro-F1 lớn nhất
Early stopping / checkpoint giữ lại	Patience 3 / 1
DataLoader workers	0
Gradient checkpointing / torch compile	Tắt / tắt

Bảng 4.1: Cấu hình huấn luyện PhoBERT.

Epoch	Train loss*	Eval loss	Accuracy	Macro-F1
1	0.833146	0.404629	0.880493	0.868397
2	0.643082	0.354720	0.894346	0.885470
3	0.458360	0.338845	0.900332	0.891702
4	0.335830	0.363673	0.904344	0.897606
5	0.263415	0.371614	0.904751	0.897426
6	0.205018	0.413617	0.904407	0.897382
7	0.191186	0.432415	0.905190	0.898581

Bảng 4.2: Loss và chỉ số phân loại qua bảy epoch. Train loss là giá trị Trainer ghi tại mốc tương ứng, không phải trung bình toàn bộ epoch.

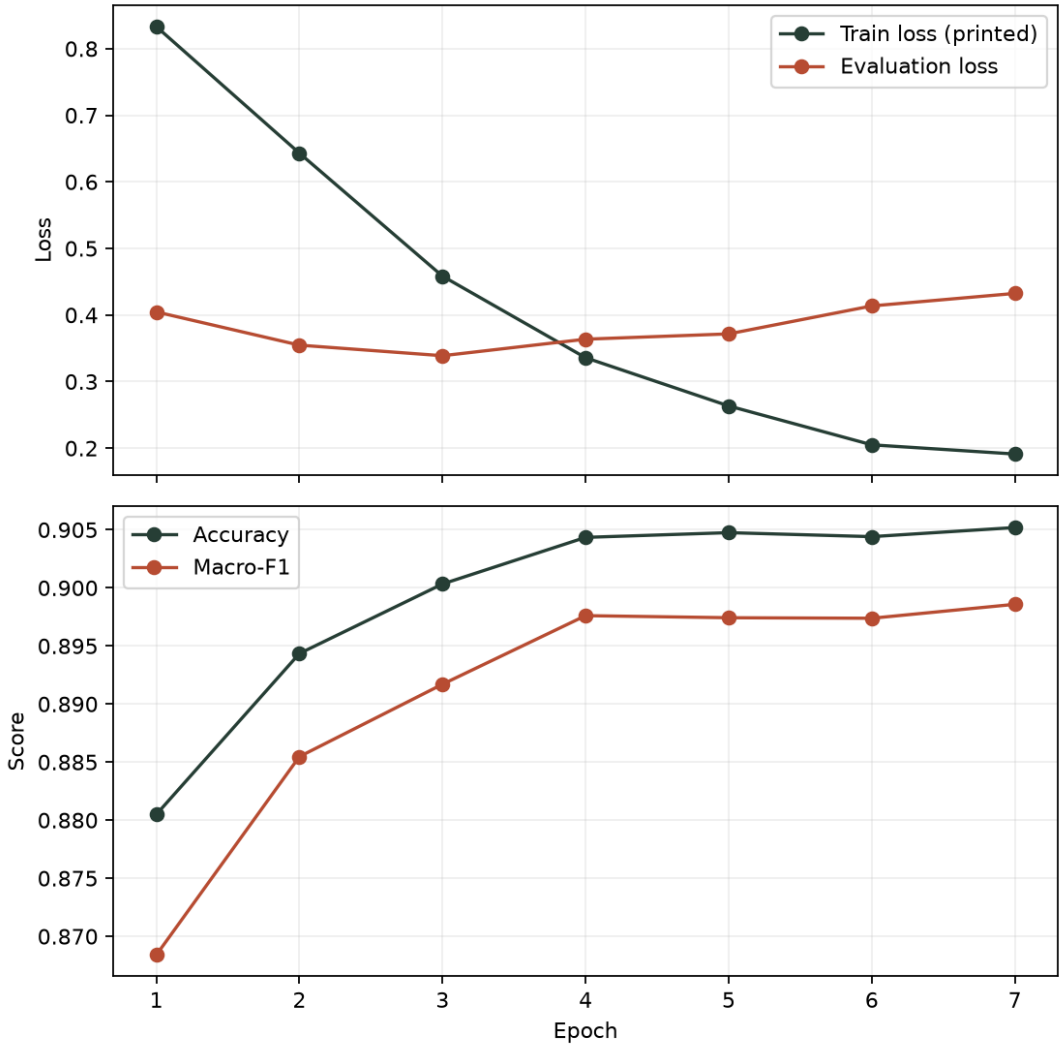
Macro-F1 tăng từ 0,868397 ở epoch 1 lên 0,898581 ở epoch 7. Mức tăng lớn nhất nằm ở những epoch đầu; từ epoch 4 trở đi, chỉ số dao động trong một khoảng hẹp. Epoch 7 đạt macro-F1 cao nhất trong bảy mốc và được chọn làm kết quả cuối.

Đường loss cho thấy một diễn biến khác: eval loss giảm đến epoch 3, đạt 0,338845, rồi tăng lên 0,432415 ở epoch 7, trong khi train loss tiếp tục giảm. Điều này gợi ý mô hình bắt đầu khớp tập huấn luyện nhiều hơn nhưng chất lượng xác suất trên tập đánh giá không còn cải thiện tương ứng. Một số dự đoán sai với độ tự tin cao có thể làm loss tăng dù số nhãn dự đoán đúng vẫn nhích lên. Đây cũng là lý do cần xem thêm NLL và ECE ở bước hiệu chỉnh, thay vì chỉ dựa vào accuracy.

4.4 Kết quả trước hiệu chỉnh

Với quy tắc chọn lớp có logit lớn nhất, mô hình đạt accuracy 90,52%, weighted-F1 0,905308 và macro-F1 0,898581 trên 31.906 bài đánh giá. Chênh lệch giữa weighted-F1 và macro-F1 cho thấy kết quả ở một số lớp nhỏ hoặc khó phân biệt còn thấp hơn mặt bằng chung.

Thể thao đạt F1 0,9901, còn Dân tộc – Tôn giáo đạt 0,7796 với 503 bài đánh giá. Các chuyên



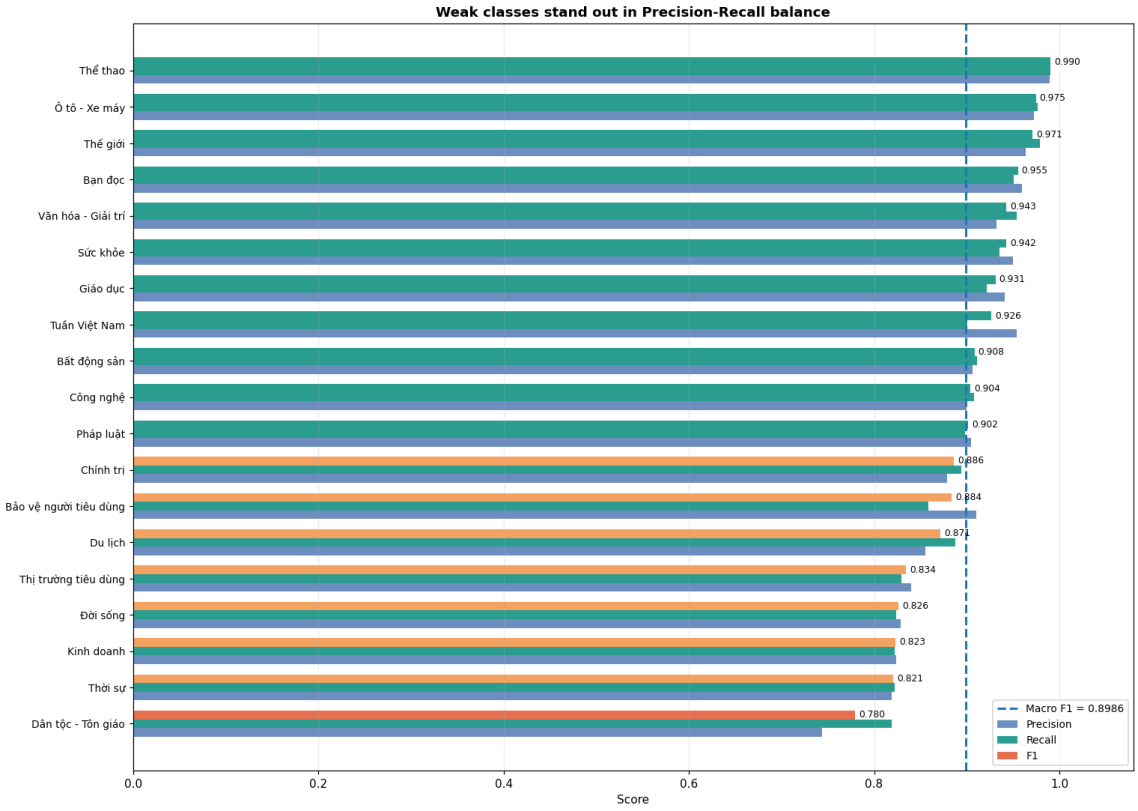
Hình 4.1: Diễn biến loss, accuracy và macro-F1 theo epoch.

mục Kinh doanh, Thời sự, Đời sống và Thị trường tiêu dùng nằm quanh mức 0,82–0,83. Vì đây đều là những nhóm có nội dung rộng hoặc gần với các nhóm khác, phần phân tích lỗi sẽ xem xét cụ thể các cặp nhầm lẫn, thay vì chỉ xếp hạng chuyên mục theo F1.

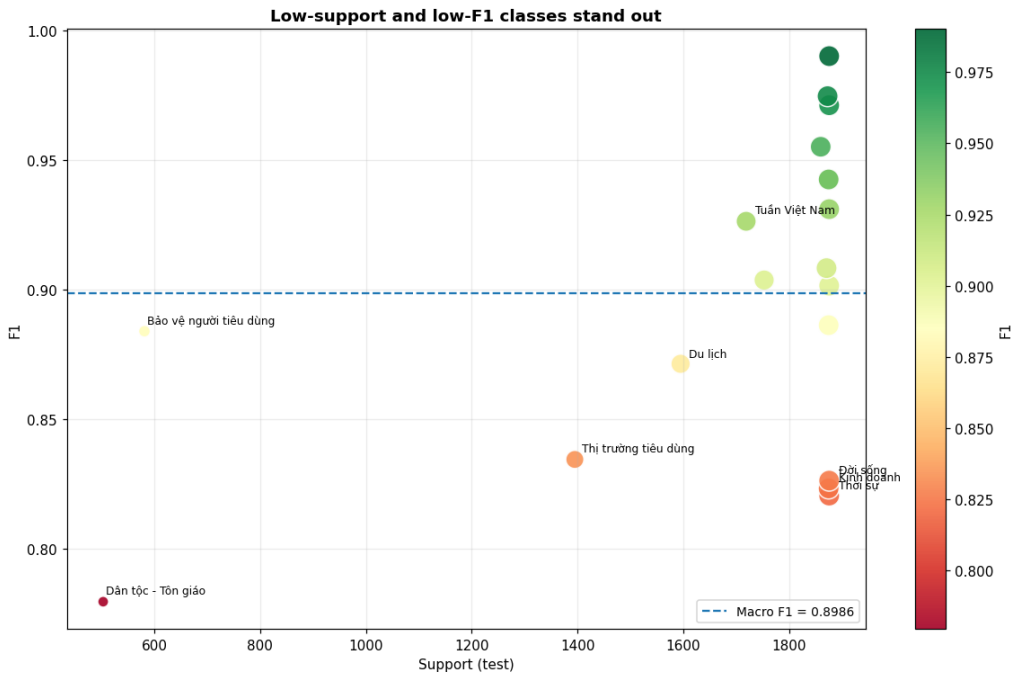
4.5 Lưu mô hình và kiểm tra suy luận

Trọng số được lưu dưới dạng safetensors, kèm cấu hình kiến trúc, tokenizer, ánh xạ nhãn và thông số tiền xử lý. Nhờ đó, đầu vào khi suy luận có thể được chuẩn bị theo cùng cách đã dùng trong huấn luyện.

Ba bài lấy từ dữ liệu nguồn được dùng để kiểm tra nhanh quá trình nạp mô hình và dự đoán. Phép thử này chỉ kiểm tra đường suy luận; chất lượng phân loại vẫn được đánh giá trên tập 31.906 bài. Sau calibration, gói mô hình được bổ sung temperature, ngưỡng theo lớp và các tệp kết quả tương ứng.



Hình 4.2: Precision, recall và F1 theo chuyên mục trước hiệu chỉnh.



Hình 4.3: Quan hệ giữa số mẫu đánh giá và F1 của từng chuyên mục.

Chương 5

Hiệu chỉnh xác suất và ngưỡng phân loại

5.1 Temperature scaling

Accuracy và F1 phản ánh nhãn được chọn, nhưng chưa mô tả đầy đủ mức độ tin cậy của xác suất dự đoán. Khi mô hình chọn sai một nhãn với xác suất rất cao, sai lệch đó thể hiện rõ hơn qua NLL. Temperature scaling được dùng để điều chỉnh độ sắc của phân bố xác suất mà vẫn giữ nguyên thứ tự giữa các lớp.

Với vector logits z_i, xác suất sau hiệu chỉnh là

$$p_{i,c}(T) = \frac{\exp(z_{i,c}/T)}{\sum_{k=1}^{K} \exp(z_{i,k}/T)}, \qquad T > 0.$$

Tham số T được tìm bằng cách tối thiểu hóa negative log-likelihood trên tập đánh giá:

$$T^* = \arg\min_T -\frac{1}{N} \sum_{i=1}^{N} \log p_{i,y_i}(T).$$

Thuật toán sử dụng `scipy.optimize.minimize_scalar` với phương pháp bounded, giới hạn T trong khoảng $[0{,}5; 5{,}0]$. Giá trị tìm được là $T^* = 1{,}4583027$. Vì T lớn hơn 1, phân bố xác suất sau hiệu chỉnh mềm hơn ban đầu. Thứ tự logits không đổi nên temperature scaling riêng lẻ không làm thay đổi lớp dự đoán.

Độ phù hợp giữa độ tự tin và tỷ lệ dự đoán đúng được đo thêm bằng ECE, sử dụng 15 khoảng độ tin cậy:

$$\mathrm{ECE} = \sum_{b=1}^{15} \frac{|B_b|}{N} \left|\mathrm{acc}(B_b) - \mathrm{conf}(B_b)\right|.$$

Sau hiệu chỉnh, NLL giảm từ 0,3828 xuống 0,3237 và ECE giảm từ 0,0608 xuống 0,0218. Như vậy, trên dữ liệu dùng để tìm T, xác suất dự đoán phù hợp hơn với tỷ lệ đúng quan sát được. Khả năng giữ mức cải thiện này trên tập độc lập vẫn cần được đánh giá riêng.

5.2 Tìm ngưỡng cho từng chuyên mục

Temperature scaling điều chỉnh độ tự tin chung. Để thay đổi cách lựa chọn giữa các chuyên mục, mỗi lớp được gán thêm một ngưỡng τ_c:

$$\widehat{y}_i = \arg\max_c \frac{p_{i,c}(T^*)}{\tau_c}.$$

Ngưỡng lớn hơn 1 làm giảm điểm của lớp, còn ngưỡng nhỏ hơn 1 làm tăng khả năng lớp đó được chọn. Các tỷ số dùng để so sánh thứ hạng; đầu ra cuối cùng vẫn là một chuyên mục cho mỗi bài báo.

Ban đầu, tất cả ngưỡng bằng 1. Quá trình tìm kiếm gồm ba lượt qua 19 lớp. Ở mỗi lớp, thuật toán thử 200 giá trị cách đều trong khoảng 0,3 đến 3,5, giữ nguyên các ngưỡng còn lại và chọn giá trị làm F1 của lớp đang xét tăng hơn 10^{-5}. Cách tìm này tối ưu lần lượt từng lớp nên không bảo đảm đạt macro-F1 lớn nhất trên toàn bộ không gian ngưỡng.

Điểm cần chú ý là quyết định giữa các lớp có liên hệ với nhau. Khi giảm ngưỡng Pháp luật, một phần bài trước đó được chọn là Thời sự có thể chuyển sang Pháp luật. Vì vậy, đánh giá hiệu chỉnh cần xem đồng thời precision, recall và các cặp nhầm lẫn, thay vì chỉ nhìn vào mức tăng F1 của một lớp.

5.3 Kết quả sau hiệu chỉnh

Chỉ số	Trước hiệu chỉnh	Sau hiệu chỉnh	Chênh lệch
Accuracy	0.905190	0.908638	+0.003448
Weighted-F1	0.905308	0.908457	+0.003149
Macro-F1	0.898581	0.901626	+0.003046

Bảng 5.1: Kết quả trước và sau khi áp dụng temperature scaling kết hợp ngưỡng theo lớp.

Accuracy tăng từ 90,5190% lên 90,8638%, tương đương 0,3448 điểm phần trăm; macro-F1 tăng từ 0,898581 lên 0,901626. Mức tăng accuracy đến từ bước thay đổi ngưỡng, vì temperature scaling không làm đổi thứ tự lớp.

Pháp luật là trường hợp thể hiện rõ đánh đổi sau hiệu chỉnh. Với ngưỡng 0,3965, F1 tăng từ khoảng 0,9016 lên 0,9093. Recall tăng từ 0,8982 lên 0,9622, nhưng precision giảm từ 0,9049 xuống 0,8620. Mô hình nhận ra thêm nhiều bài thuộc Pháp luật, đồng thời cũng đưa nhầm thêm bài từ chuyên mục khác vào lớp này.

Ở Thời sự, ngưỡng 1,5060 đi theo chiều ngược lại: precision tăng nhưng recall giảm còn 0,7772. Do đó, mức cải thiện chỉ số chung không có nghĩa mọi lớp đều được lợi theo cùng một cách. Việc chọn ngưỡng còn phụ thuộc yêu cầu ưu tiên bỏ sót ít hay dự đoán nhầm ít ở từng chuyên mục.

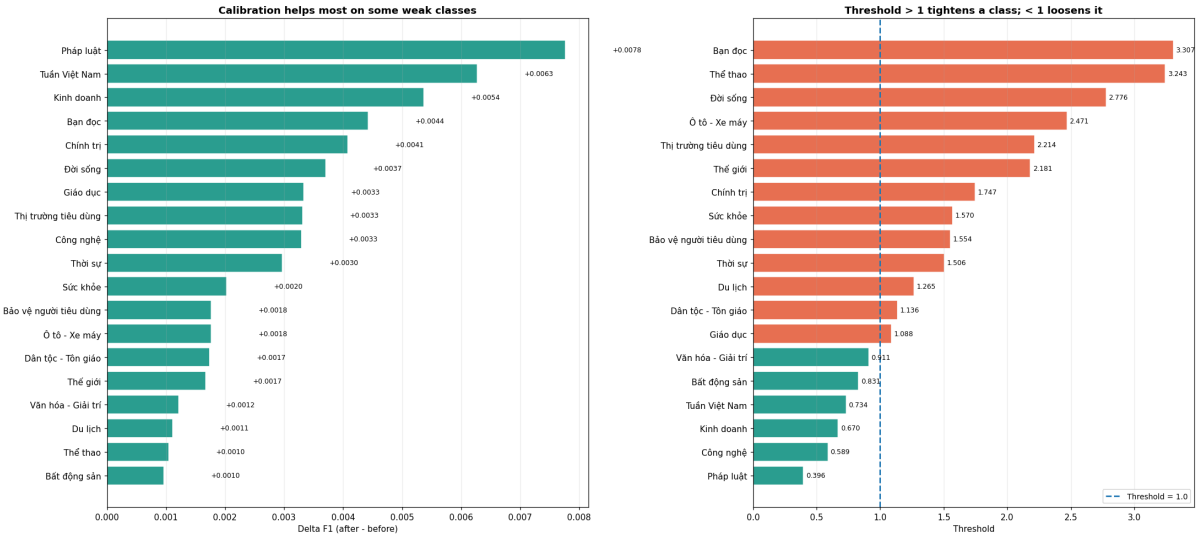
5.4 Lưu tham số hiệu chỉnh

Temperature, ngưỡng của từng lớp và thông số tìm kiếm được lưu trong `thresholds.json`. Tệp `label_config.json` chứa đường dẫn đến cấu hình ngưỡng và cờ sử dụng calibration. Các chỉ số tổng hợp, báo cáo theo lớp và ma trận nhầm lẫn được xuất thành JSON cùng gói mô hình.

Ma trận nhầm lẫn trong các tệp JSON phản ánh kết quả sau hiệu chỉnh; hình ở phần đánh giá ban đầu phản ánh kết quả trước hiệu chỉnh. Phân biệt hai thời điểm này là cần thiết khi đối chiếu số lỗi và chỉ số của từng lớp.

Chuyên mục	Số mẫu	F1 trước	F1 sau	Ngưỡng
Bạn đọc	1860	0.9552	0.9596	3.3070
Bảo vệ người tiêu dùng	581	0.8840	0.8857	1.5543
Bất động sản	1871	0.9083	0.9093	0.8307
Chính trị	1875	0.8863	0.8904	1.7472
Công nghệ	1753	0.9037	0.9070	0.5894
Du lịch	1595	0.8714	0.8725	1.2648
Dân tộc - Tôn giáo	503	0.7796	0.7813	1.1362
Giáo dục	1876	0.9311	0.9344	1.0879
Kinh doanh	1875	0.8233	0.8286	0.6698
Pháp luật	1876	0.9016	0.9093	0.3965
Sức khỏe	1875	0.9425	0.9445	1.5704
Thế giới	1876	0.9712	0.9729	2.1814
Thể thao	1876	0.9901	0.9912	3.2427
Thị trường tiêu dùng	1395	0.8345	0.8378	2.2136
Thời sự	1876	0.8205	0.8235	1.5060
Tuần Việt Nam	1719	0.9264	0.9327	0.7342
Văn hóa - Giải trí	1875	0.9425	0.9438	0.9111
Ô tô - Xe máy	1873	0.9747	0.9765	2.4709
Đời sống	1876	0.8263	0.8300	2.7764

Bảng 5.2: F1 trước, F1 sau và ngưỡng của 19 chuyên mục. F1 trước được ghi với bốn chữ số thập phân; ngưỡng và F1 sau lấy từ kết quả xuất mô hình.



Hình 5.1: Thay đổi F1 và ngưỡng sau hiệu chỉnh theo chuyên mục.

Chương 6

Phân tích lỗi và kết luận

6.1 Những chuyên mục còn khó phân biệt

Sau hiệu chỉnh, Dân tộc – Tôn giáo vẫn có F1 thấp nhất, đạt 0,7813, với precision 0,7372 và recall 0,8310. Mức thay đổi nhỏ so với trước hiệu chỉnh cho thấy chỉ điều chỉnh ngưỡng chưa đủ để cải thiện đáng kể lớp này. Việc khảo sát thêm các bài bị nhầm và độ đa dạng của mẫu huấn luyện sẽ giúp xác định nguyên nhân cụ thể hơn.

Nhãn thực	Dự đoán	Số lỗi	Tỷ lệ trong lớp
Thời sự	Pháp luật	236	12.58%
Thị trường tiêu dùng	Kinh doanh	127	9.10%
Đời sống	Du lịch	125	6.66%
Kinh doanh	Bất động sản	64	3.41%
Bất động sản	Kinh doanh	60	3.21%
Chính trị	Dân tộc - Tôn giáo	56	2.99%
Thời sự	Chính trị	55	2.93%
Đời sống	Kinh doanh	54	2.88%

Bảng 6.1: Tám cặp nhầm lẫn có số lỗi lớn nhất sau hiệu chỉnh. Tỷ lệ được tính trên số bài thật của chuyên mục ở cột đầu.

Cặp Thời sự → Pháp luật có 236 lỗi, chiếm 12,58% số bài Thời sự trong tập đánh giá. Đây cũng là cặp cần xem xét khi đánh giá tác động của ngưỡng Pháp luật thấp. Hai nhóm lỗi lớn tiếp theo là Thị trường tiêu dùng → Kinh doanh với 127 bài và Đời sống → Du lịch với 125 bài.

Các cặp này cho thấy việc đọc kết quả theo từng chuyên mục có giá trị hơn chỉ nhìn vào accuracy tổng. Để cải thiện tiếp, cần đọc những bài thuộc đúng các nhóm lỗi trên, xem nội dung nào khiến mô hình chuyển lớp và đối chiếu với cách phân chuyên mục của nguồn dữ liệu. Ma trận nhầm lẫn giúp chọn nơi cần khảo sát, nhưng tự nó chưa đủ để kết luận nguyên nhân nằm ở mô hình hay ở nhãn.

6.2 Giới hạn thực nghiệm

Giới hạn quan trọng nhất là cách sử dụng tập đánh giá. Cùng 31.906 bài đã tham gia chọn checkpoint, tìm temperature và tìm ngưỡng. Vì vậy, các chỉ số sau hiệu chỉnh phản ánh kết quả trên dữ liệu đã được dùng để lựa chọn tham số, chưa phải ước lượng trên một tập test độc lập. Một thiết kế ba tập, trong đó validation dùng cho lựa chọn và calibration còn test chỉ dùng ở cuối, sẽ cho phép đánh giá rõ hơn khả năng khái quát hóa.

Thực nghiệm sử dụng một cách chia với seed 42 nên chưa mô tả được độ biến thiên giữa các lần chạy. Dữ liệu cũng chưa được loại trùng URL hoặc văn bản trước khi chia, chưa chia theo

thời gian và chưa lưu ID của từng tập. Ngoài ra, revision cùng checksum của dữ liệu nguồn chưa được ghi lại trong lần huấn luyện, làm hạn chế khả năng tái lập chính xác từng bản ghi.

Quá trình nạp lại checkpoint có cảnh báo về tên khóa LayerNorm dạng beta/gamma và weight/bias. Vấn đề tương thích này cần được kiểm tra trong một lần tái lập đầy đủ. Các kiểm tra hiện tại xác nhận chỉ số sau hiệu chỉnh khớp với ma trận nhầm lẫn đã lưu; do chưa có raw logits và dự đoán từng bài, NLL/ECE chưa được tính lại độc lập từ đầu ra mô hình.

6.3 Kết luận

PhoBERT đạt accuracy 90,52% và macro-F1 0,8986 trên tập đánh giá của thực nghiệm. Temperature scaling làm giảm NLL và ECE; kết hợp ngưỡng theo lớp đưa accuracy lên 90,86% và macro-F1 lên 0,9016. Hiệu chỉnh mang lại mức tăng vừa phải về chỉ số chung, nhưng ảnh hưởng rõ hơn đến sự cân bằng giữa precision và recall của một số chuyên mục.

Kết quả còn hạn chế ở các lớp Dân tộc – Tôn giáo, Thời sự, Kinh doanh và những cặp có nội dung gần nhau. Hướng đánh giá tiếp theo cần ưu tiên chia tập độc lập sau khi xử lý trùng, rồi chọn checkpoint và tham số calibration trên validation. Trên cơ sở đó mới có thể xác định mức cải thiện giữ được khi gặp bài báo chưa tham gia quá trình lựa chọn mô hình.

Báo cáo sử dụng AI được đính kèm tại tệp `AI_USE_REPORT.md` trong repository.

Chương A

Thực nghiệm bổ sung trên tiêu đề

Một bộ đối chứng gồm 7.600 tiêu đề được chọn cân bằng giữa 19 lớp, với train/validation/test lần lượt có 5.320/1.140/1.140 mẫu. Các mô hình gồm TF-IDF từ, TF-IDF ký tự và đặc trưng PhoBERT đóng băng kết hợp Logistic Regression. Tham số của bộ phân loại được chọn theo macro-F1 trên validation trước khi đánh giá test.

Mô hình	Accuracy	Macro-F1
Majority	5,26%	0,0053
TF-IDF từ + Linear SVM	66,32%	0,6582
TF-IDF ký tự + Linear SVM	67,28%	0,6676
PhoBERT đóng băng + Logistic Regression	77,54%	0,7738

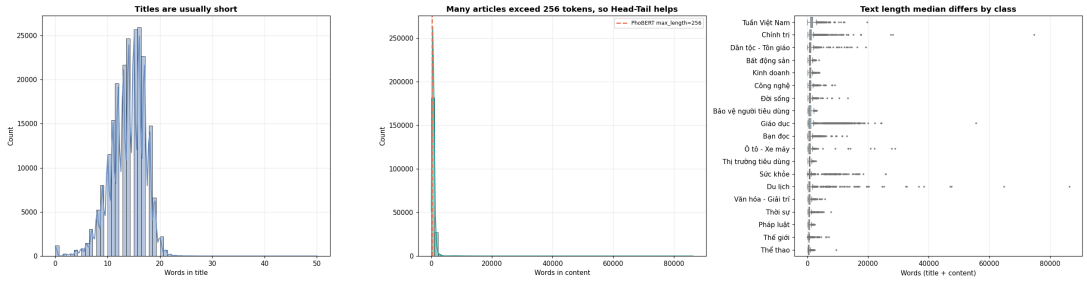
Bảng A.1: Kết quả trên 1.140 tiêu đề thuộc tập test của bộ đối chứng.

PhoBERT đóng băng đạt kết quả cao nhất trong ba cách biểu diễn đã khảo sát. Tuy nhiên, mức 77,54% không so sánh trực tiếp với 90,52% của mô hình fine-tune vì hai thực nghiệm khác đầu vào, số mẫu, phân bố lớp và cách sử dụng tập đánh giá. Chi tiết được trình bày trong `notebooks/text_baselines.ipynb` và `reports/text/baselines.pdf`.

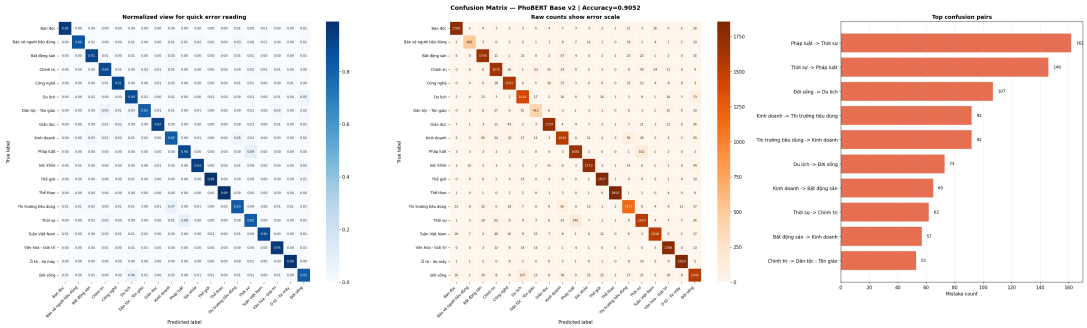
Chương B

Biểu đồ bổ sung

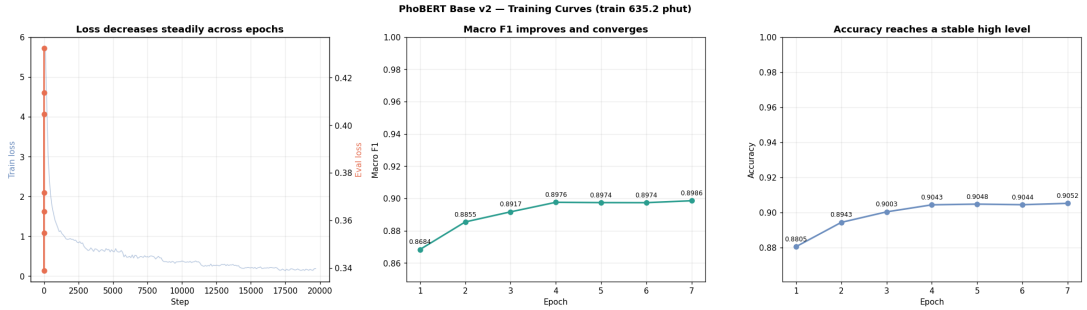
Các hình dưới đây bổ sung thống kê độ dài, ma trận nhầm lẫn trước hiệu chỉnh và đường cong huấn luyện. Cùng với bốn hình trong các chương trước, chúng tạo thành bộ bảy hình được lưu trong notebook thực nghiệm.



Hình B.1: Độ dài tiêu đề, nội dung và phân bố theo chuyên mục. Mốc 256 ở hình này có đơn vị từ; độ dài token đã được khảo sát riêng sau tiền xử lý.



Hình B.2: Ma trận nhầm lẫn trước hiệu chỉnh: tỷ lệ theo hàng, số bài và các cặp có nhiều lỗi.



Hình B.3: Đường cong lưu trong notebook. Ở ô loss, mốc của đường đánh giá là epoch còn mốc của đường huấn luyện là step; hình ở Chương 4 đã thống nhất lại trục epoch.

Chương C

Mã nguồn và đối chiếu kết quả

C.1 Môi trường và cấu hình

Mô hình được huấn luyện trên Kaggle với hai GPU Tesla T4, mỗi GPU có khoảng 14,6 GiB bộ nhớ. Môi trường sử dụng Python 3.12.12 và PyTorch 2.10.0+cu128. Các thư viện chính gồm Transformers, Accelerate, scikit-learn và PyVi. FP16 được dùng để giảm bộ nhớ và tận dụng khả năng tính toán của T4.

Đường dẫn dữ liệu, thư mục lưu mô hình và các tham số được khai báo tập trung. Dữ liệu được tải từ Hugging Face khi chưa có bản cục bộ. Giới hạn số mẫu mỗi lớp được đặt bằng 0 để sử dụng toàn bộ dữ liệu đã nạp.

C.2 Tệp thực nghiệm và kết quả

Mô hình được huấn luyện trên Kaggle từ ngày 26 đến 27/4/2026 theo giờ Việt Nam. Notebook thực nghiệm gồm 36 cell, trong đó có 25 cell code đã chạy. File `notebooks/text.ipynb` giữ đầy đủ mã nguồn, kết quả và hình của lần chạy này. Mã nguồn các cell khớp với notebook trong repository ứng dụng tại commit `e171688` [2].

Gói mô hình gồm trọng số khoảng 540 MB, tokenizer và cấu hình phân loại. Danh sách tệp cùng SHA256 được lưu tại `reports/text/reuse-manifest.json`. Dữ liệu thô, trọng số và cache được giữ ngoài Git; các bảng, hình và kết quả tổng hợp nằm trong thư mục báo cáo.

Lệnh `python -m text_reuse.analysis` kiểm tra chữ ký các tệp và tính lại accuracy, macro-F1, weighted-F1 cùng precision/recall/F1 theo lớp từ ma trận nhầm lẫn sau hiệu chỉnh. Các giá trị trước hiệu chỉnh, bảng epoch và NLL/ECE được đối chiếu với kết quả đã lưu trong notebook. Để xuất lại bảng và biên dịch báo cáo, sử dụng:

```
python -m text_reuse.analysis --export
python scripts/build_report.py --track text
```

Notebook chứa toàn bộ quy trình huấn luyện; khi chạy lại cần thiết lập đường dẫn ở Section 0.3 và xóa cache nếu thay dữ liệu. Bộ mã kiểm tra và tài liệu nguồn được trình bày chi tiết trong `docs/TEXT_REUSE.md`.

Tài liệu tham khảo

[1] Huỳnh Thành Đạt. *VietnamNet News*. https://huggingface.co/datasets/dathuynh1108/vietnamnet-news.

[2] Huỳnh Thành Đạt. *PhoBERT News Classification App*. Mã nguồn và gói mô hình tại commit e171688. https://github.com/dathuynh1108/phobert-news-classification-app.

[3] D. Q. Nguyen và A. T. Nguyen. *PhoBERT: Pre-trained language models for Vietnamese*. Findings of EMNLP, 2020. https://huggingface.co/vinai/phobert-base-v2.

[4] PyVi. Vietnamese word segmentation. https://pypi.org/project/pyvi/.

[5] Hugging Face. Transformers: Trainer, TrainingArguments và sequence classification. https://huggingface.co/docs/transformers/.

[6] PyTorch. CrossEntropyLoss. https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html.